
Algorithm Extreme Learning Machines

Load dataset $\{\mathbf{x}_i, \mathbf{y}_i\}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{y} \in \mathbb{R}^m$ $i = 1, 2, \dots, N$
Choose N_h $\triangleright$ No. of neurons in hidden layer
Define $h()$ $\triangleright$ hidden layer activation function
Initialize $\mathbf{W}^{(1)} \in \mathbb{R}^{N_h \times n}$, $\mathbf{b}^{(1)} \in \mathbb{R}^{N_h \times 1}$ randomly $\triangleright$ hidden weights and bias
Compute $\mathbf{a}_i^{(1)} = h(\mathbf{W}^{(1)}\mathbf{x}_i + \mathbf{b}^{(1)})$, $i = 1, 2, \dots, N$
Frame coefficient matrix $\mathbf{H} = \{\mathbf{a}_i\}$, $i = 1, 2, \dots, N$ $\triangleright N_h \times N$ matrix
if Include bias in output layer **then**
 $\mathbf{H} := [\mathbf{H}^T | \{1\}^{N_h \times 1}]^T$
end if
Frame output matrix $\mathbf{Y} = \{bmy_i\}$, $i = 1, 2, \dots, N$ $\triangleright m \times N$ matrix
Compute $\mathbf{W}^{(2)} = \mathbf{Y}\mathbf{H}^\dagger$ $\triangleright$ output weight matrix, † pseudo-inverse
